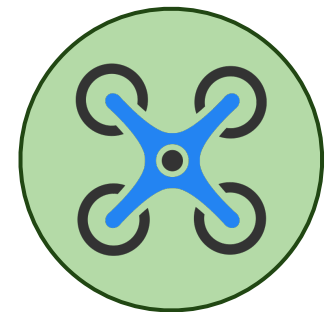
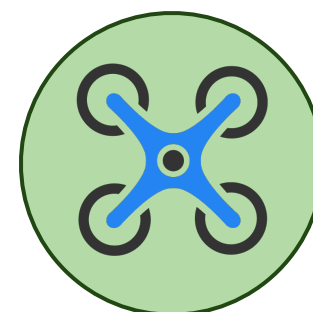
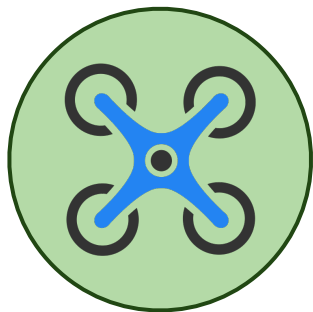
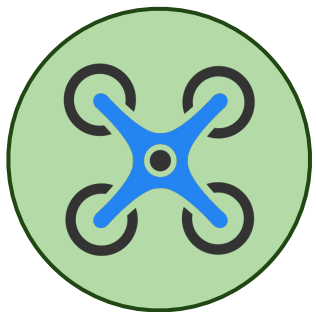

Deadlock Resolution of Multi-robot Systems using Neural Logic Machines

Ruiyang Wang

Deadlock situation for multi-robot systems



Formal Definition of Deadlocks

Consider a set of M homogeneous robots $\mathbb{M} = \{m^i | i = 1, \dots, M\}$ with discrete linear dynamics

$$x_{t+1}^i = \mathbf{A}x_t^i + \mathbf{B}u_t^i$$

where $x_t^i = \begin{bmatrix} p_t^i \\ v_t^i \end{bmatrix}$ is the state (position p_t^i and velocity v_t^i) and u_t^i is the control input (acceleration).

The optimal control input $u_t^i, u_{t+1}^i, \dots, u_{t+T}^i$ of each robot i starting at timestep t for a time horizon T is then calculated distributively from a constrained optimization problem:

$$\begin{aligned} u_t^i, u_{t+1}^i, \dots, u_{t+T}^i &= \arg \min_{u_{t+\tau}^i \in U} \sum_{\tau=0}^T \|x_{t+\tau+1}^i - x_{target}^i\|_2^2 + \lambda \|u_{t+\tau}^i\|_2^2 \\ x_{t+\tau+1}^i &= \mathbf{A}x_{t+\tau}^i + \mathbf{B}u_{t+\tau}^i \quad \mathbf{I}_2 |v_{t+\tau+1}^i| \leq v_{max} \quad \mathbf{I}_2 |u_{t+\tau+1}^i| \leq a_{max} \\ \max(\mathbf{A}_{obs}^j x_{t+\tau+1}^i - b_{obs}^j) &\geq r \quad \forall j \in \{1, \dots, N\} \\ \max(\mathbf{A}_{rob,t+\tau+1}^k x_{t+\tau+1}^i - b_{rob,t+\tau+1}^k) &\geq r \quad \forall k \in \{1, \dots, M\} \setminus i; \end{aligned}$$

Formal Definition of Deadlocks

The optimal control input $u_t^i, u_{t+1}^i, \dots, u_{t+T}^i$ of each robot i starting at timestep t for a time horizon T is then calculated distributively from a constrained optimization problem:

$$\begin{aligned}
 u_t^i, u_{t+1}^i, \dots, u_{t+T}^i &= \arg \min_{u_{t+\tau}^i \in U} \sum_{\tau=0}^T \|x_{t+\tau+1}^i - x_{target}^i\|_2^2 + \lambda \|u_{t+\tau}\|_2^2 \\
 x_{t+\tau+1}^i &= \mathbf{A}x_{t+\tau}^i + \mathbf{B}u_{t+\tau}^i \quad \mathbf{I}_2|v_{t+\tau+1}^i| \leq v_{max} \quad \mathbf{I}_2|u_{t+\tau+1}^i| \leq a_{max} \\
 \max(\mathbf{A}_{obs}^j x_{t+\tau+1}^i - b_{obs}^j) &\geq r \quad \forall j \in \{1, \dots, N\} \\
 \max(\mathbf{A}_{rob,t+\tau+1}^k x_{t+\tau+1}^i - b_{rob,t+\tau+1}^k) &\geq r \quad \forall k \in \{1, \dots, M\} \setminus i;
 \end{aligned}$$

A deadlock happens at some timestep t_{dead} when

$$\|p_{t_{dead}+1}^i - p_{t_{dead}}^i\|_2 \leq \epsilon, \quad \|p_{t_{dead}+1}^i - p_{target}^i\|_2 \geq \epsilon, \quad \forall i \in \{1, \dots, M\}$$

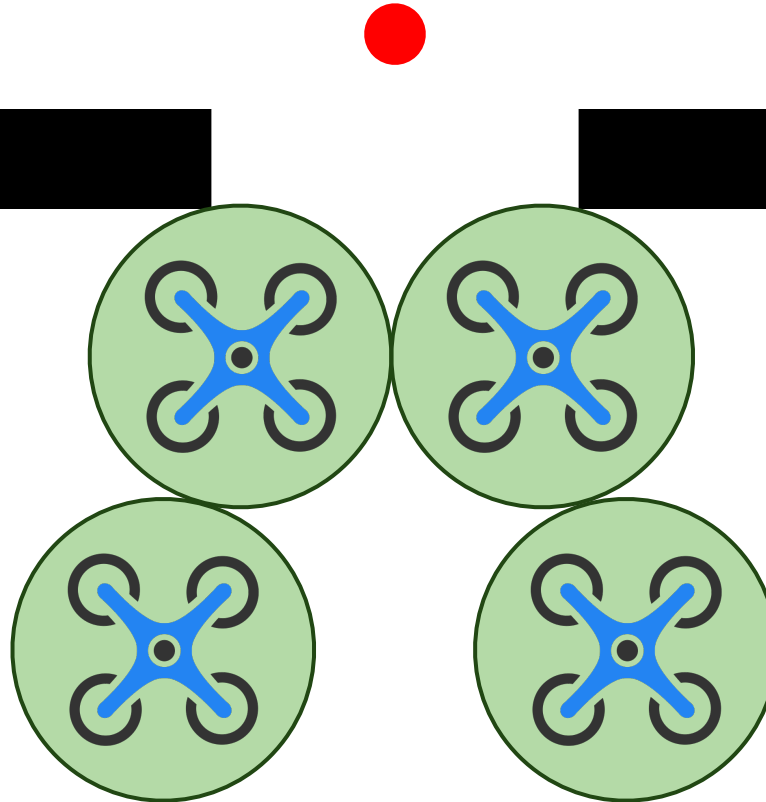
Formal Definition of Deadlocks

The optimal control input $u_t^i, u_{t+1}^i, \dots, u_{t+T}^i$ of each robot i starting at timestep t for a time horizon T is then calculated distributively from a constrained optimization problem:

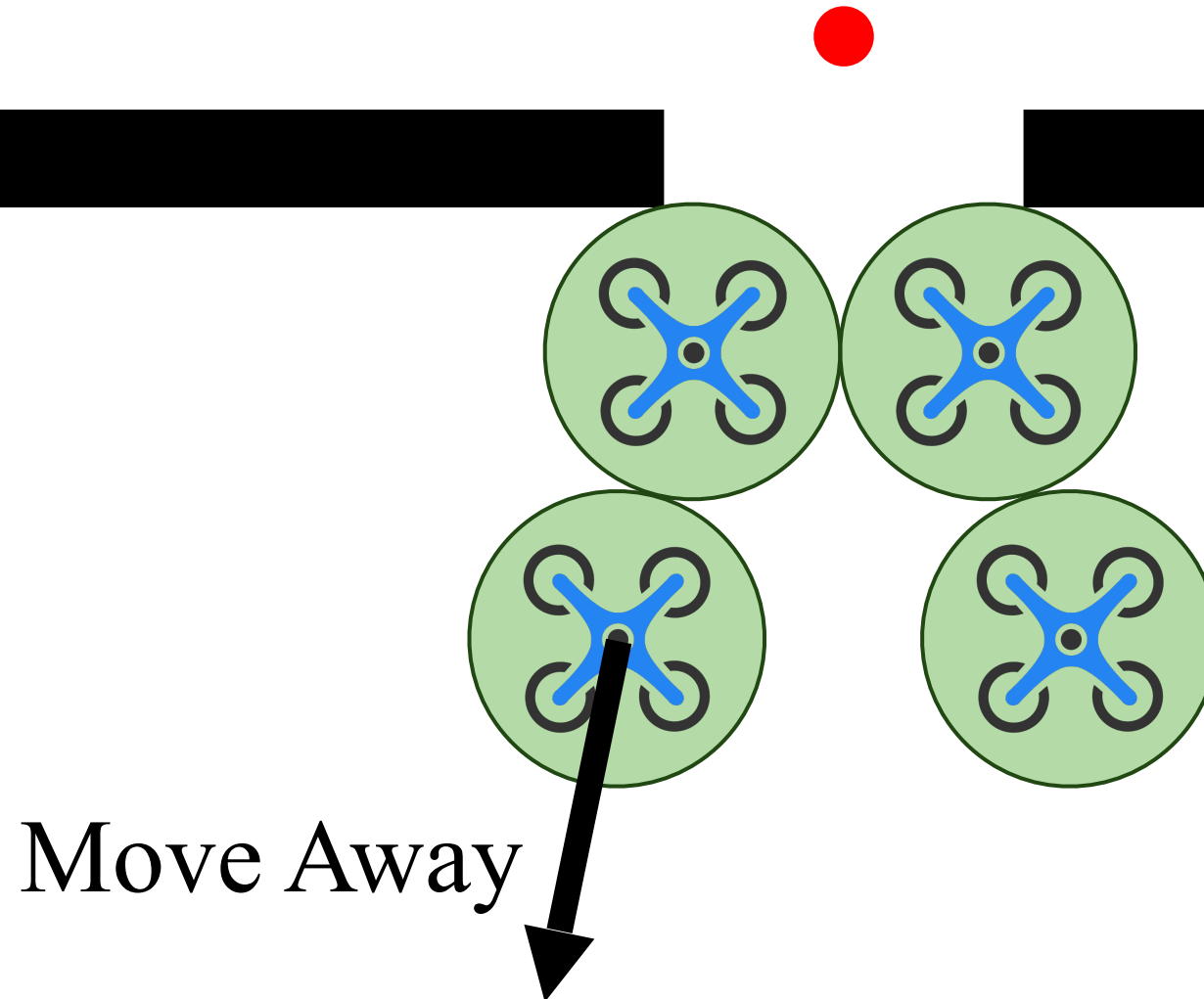
$$\begin{aligned}
 u_t^i, u_{t+1}^i, \dots, u_{t+T}^i &= \arg \min_{u_{t+\tau}^i \in U} \sum_{\tau=0}^T \|x_{t+\tau+1}^i - x_{target}^i\|_2^2 + \lambda \|u_{t+\tau}\|_2^2 \\
 x_{t+\tau+1}^i &= \mathbf{A}x_{t+\tau}^i + \mathbf{B}u_{t+\tau}^i \quad \mathbf{I}_2 |v_{t+\tau+1}^i| \leq v_{max} \quad \mathbf{I}_2 |a_{t+\tau+1}^i| \leq a_{max} \\
 \max(\mathbf{A}_{obs}^j x_{t+\tau+1}^i - b_{obs}^j) &\geq r \quad \forall j \in \{1, \dots, N\} \\
 \max(\mathbf{A}_{rob,t+\tau+1}^k x_{t+\tau+1}^i - b_{rob,t+\tau+1}^k) &\geq r \quad \forall k \in \{1, \dots, M\} \setminus i;
 \end{aligned}$$

We want to resolve the deadlock and all robots arrive at their targets as soon as possible

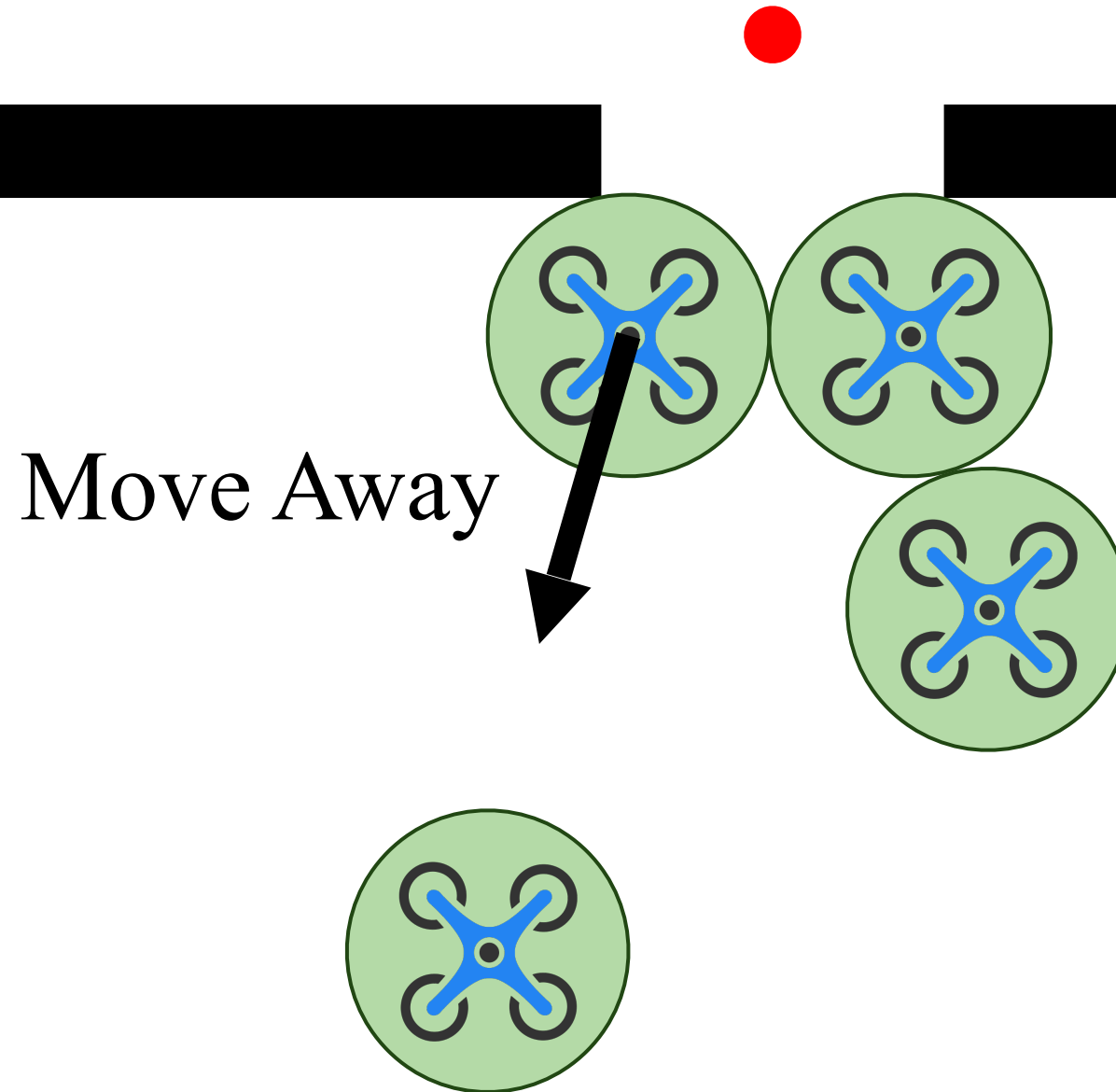
Resolution to the deadlock situation



Resolution to the deadlock situation



Resolution to the deadlock situation



Deadlocks Resolution

Algorithm 1 Greedy Algorithm for Deadlock Resolution

```

 $T_H$ ; // Full Time horizon considered
 $\mathcal{M} = \{m^1, \dots, m^M\}$ ; // List of robots initialized with active status
 $\mathcal{I}_{passive} \leftarrow \emptyset$ ; // Set of indexes of all passive robots
for  $t = 1$  to  $T_H$  do
    if deadlock detected among active robots then
        // Find the index of the robot closest to its target
         $i_{closest} \leftarrow \arg \min_{i \in \{1, \dots, M\}} \|p_t^i - p_{target}^i\|_2$ 
        Sort robots  $m^i \in \mathcal{M}$  by decreasing distance to  $m^{i_{closest}}$ 
        // Update  $m^i.\mathcal{I}_{passive}^i$  iteratively, and mark  $m^i$  as passive
        for each robot  $m^i$  in the sorted list of active robots other than  $m^{i_{closest}}$  do
             $m^i.\mathcal{I}_{passive}^i \leftarrow \mathcal{I}_{passive}$   $\mathcal{I}_{passive} = \mathcal{I}_{passive} \cup i$ 
        end
         $m^{i_{closest}}.\mathcal{I}_{passive}^{i_{closest}} \leftarrow \mathcal{I}_{passive}$ ; //  $\star$  Update  $\mathcal{I}_{passive}^{i_{closest}}$  for  $m^{i_{closest}}$   $\star$ 
    end
    if all active robots have arrived at target positions then
        // Re-activate robots based on the distance to targets
         $i_{closest} \leftarrow \arg \min_{i \in \mathcal{I}_{passive}} \|p_t^i - p_{target}^i\|_2$   $\mathcal{I}_{passive} = \mathcal{I}_{passive} \setminus i_{closest}$ 
        Set  $m^{i_{closest}}$  status to active  $m^i.\mathcal{I}_{passive}^i = m^i.\mathcal{I}_{passive}^i \setminus i_{closest} \forall i \in \mathcal{I}_{passive}$ 
    end
end

```

We start the simulation with all robots have an **active** status, whose control is optimized for:

$$u_t^i, u_{t+1}^i, \dots, u_{t+T}^i = \arg \min_{u_{t+\tau}^i \in U} \sum_{\tau=0}^{T-t} \|x_{t+\tau+1}^i - x_{target}^i\|_2^2 + \lambda \|u_{t+\tau}^i\|_2^2$$

$$x_{t+\tau+1}^i = \mathbf{A}x_{t+\tau}^i + \mathbf{B}u_{t+\tau}^i \quad \mathbf{I}_2 |v_{\tau+1+t}^i| \leq v_{max} \quad \mathbf{I}_2 |u_{\tau+1+t}^i| \leq a_{max}$$

$$\max(\mathbf{A}_{obs}^j x_{t+\tau+1}^i - b_{obs}^j) \geq r \quad \forall j \in \{1, \dots, N\}$$

$$\max(\mathbf{A}_{rob,t+\tau+1}^k x_{t+\tau+1}^i - b_{rob,t+\tau+1}^k) \geq r \quad \forall k \in \{1, \dots, M\} \setminus \{i \cup \mathcal{I}_{passive}^i\}$$

Deadlocks Resolution

Algorithm 1 Greedy Algorithm for Deadlock Resolution

```

 $T_H$ ; // Full Time horizon considered
 $\mathcal{M} = \{m^1, \dots, m^M\}$ ; // List of robots initialized with active status
 $\mathcal{I}_{passive} \leftarrow \emptyset$ ; // Set of indexes of all passive robots
for  $t = 1$  to  $T_H$  do
  if deadlock detected among active robots then
    // Find the index of the robot closest to its target
     $i_{closest} \leftarrow \arg \min_{i \in \{1, \dots, M\}} \|p_t^i - p_{target}^i\|_2$ 
    Sort robots  $m^i \in \mathcal{M}$  by decreasing distance to  $m^{i_{closest}}$ 
    // Update  $m^i.\mathcal{I}_{passive}^i$  iteratively, and mark  $m^i$  as passive
    for each robot  $m^i$  in the sorted list of active robots other than  $m^{i_{closest}}$  do
      |  $m^i.\mathcal{I}_{passive}^i \leftarrow \mathcal{I}_{passive}$   $\mathcal{I}_{passive} = \mathcal{I}_{passive} \cup i$ 
    end
     $m^{i_{closest}}.\mathcal{I}_{passive}^{i_{closest}} \leftarrow \mathcal{I}_{passive}$ ; // * Update  $\mathcal{I}_{passive}^{i_{closest}}$  for  $m^{i_{closest}}$  */
  end
  if all active robots have arrived at target positions then
    // Re-activate robots based on the distance to targets
     $i_{closest} \leftarrow \arg \min_{i \in \mathcal{I}_{passive}} \|p_t^i - p_{target}^i\|_2$   $\mathcal{I}_{passive} = \mathcal{I}_{passive} \setminus i_{closest}$ 
    Set  $m^{i_{closest}}$  status to active  $m^i.\mathcal{I}_{passive}^i = m^i.\mathcal{I}_{passive}^i \setminus i_{closest} \forall i \in \mathcal{I}_{passive}$ 
  end
end

```

When deadlock is detected, we use a greedy algorithm to **select** some agents to be **passive**, whose control is optimized for:

$$u_t^i, u_{t+1}^i, \dots, u_{t+T}^i = \arg \min_{u_{t+\tau}^i \in U} \sum_{\tau=0}^T \|u_{t+\tau}^i\|_2^2$$

$$x_{t+\tau+1}^i = \mathbf{A}x_{t+\tau}^i + \mathbf{B}u_{t+\tau}^i \quad \mathbf{I}_2|v_{\tau+1+t}^i| \leq v_{max} \quad \mathbf{I}_2|u_{\tau+1+t}^i| \leq a_{max}$$

$$\max(\mathbf{A}_{obs}^j x_{t+\tau+1}^i - b_{obs}^j) \geq r \quad \forall j \in \{1, \dots, N\}$$

$$\max(\mathbf{A}_{rob,t+\tau+1}^k x_{t+\tau+1}^i - b_{rob,t+\tau+1}^k) \geq r \quad \forall k \in \{1, \dots, M\} \setminus \{i \cup \mathcal{I}_{passive}\}$$

Now, the only objective for the passive robots is to avoid other robots

Deadlocks Resolution

Algorithm 1 Greedy Algorithm for Deadlock Resolution

```

 $T_H$ ; // Full Time horizon considered
 $\mathcal{M} = \{m^1, \dots, m^M\}$ ; // List of robots initialized with active status
 $\mathcal{I}_{passive} \leftarrow \emptyset$ ; // Set of indexes of all passive robots
for  $t = 1$  to  $T_H$  do
  if deadlock detected among active robots then
    // Find the index of the robot closest to its target
     $i_{closest} \leftarrow \arg \min_{i \in \{1, \dots, M\}} \|p_t^i - p_{target}^i\|_2$ 
    Sort robots  $m^i \in \mathcal{M}$  by decreasing distance to  $m^{i_{closest}}$ 
    // Update  $m^i.\mathcal{I}_{passive}^i$  iteratively, and mark  $m^i$  as passive
    for each robot  $m^i$  in the sorted list of active robots other than  $m^{i_{closest}}$  do
       $m^i.\mathcal{I}_{passive}^i \leftarrow \mathcal{I}_{passive}$   $\mathcal{I}_{passive} = \mathcal{I}_{passive} \cup i$ 
    end
     $m^{i_{closest}}.\mathcal{I}_{passive}^{i_{closest}} \leftarrow \mathcal{I}_{passive}$ ; // Update  $\mathcal{I}_{passive}^{i_{closest}}$  for  $m^{i_{closest}}$  */
  end
  if all active robots have arrived at target positions then
    // Re-activate robots based on the distance to targets
     $i_{closest} \leftarrow \arg \min_{i \in \mathcal{I}_{passive}} \|p_t^i - p_{target}^i\|_2$   $\mathcal{I}_{passive} = \mathcal{I}_{passive} \setminus i_{closest}$ 
    Set  $m^{i_{closest}}$  status to active  $m^i.\mathcal{I}_{passive}^i = m^i.\mathcal{I}_{passive}^i \setminus i_{closest} \forall i \in \mathcal{I}_{passive}$ 
  end
end

```

We start the simulation with all robots have an **active** status, whose control is optimized for:

$$u_t^i, u_{t+1}^i, \dots, u_{t+T}^i = \arg \min_{u_{t+\tau}^i \in U} \sum_{\tau=0}^T \|x_{\tau+1+t}^i - x_{target}^i\|_2^2 + \lambda \|u_{t+\tau}\|_2^2$$

$$x_{t+\tau+1}^i = \mathbf{A}x_{t+\tau}^i + \mathbf{B}u_{t+\tau}^i \quad \mathbf{I}_2 |v_{\tau+1+t}^i| \leq v_{max} \quad \mathbf{I}_2 |u_{\tau+1+t}^i| \leq a_{max}$$

$$\max(\mathbf{A}_{obs}^j x_{t+\tau+1}^i - b_{obs}^j) \geq r \quad \forall j \in \{1, \dots, N\}$$

$$\max(\mathbf{A}_{rob,t+\tau+1}^k x_{t+\tau+1}^i - b_{rob,t+\tau+1}^k) \geq r \quad \forall k \in \{1, \dots, M\} \setminus \{i \cup \mathcal{I}_{passive}^i\}$$

With that, we can remove the constraint for collision avoidance between active and passive robots in active robots' optimization problem.

Deadlocks Resolution

Algorithm 1 Greedy Algorithm for Deadlock Resolution

```

 $T_H$ ; // Full Time horizon considered
 $\mathcal{M} = \{m^1, \dots, m^M\}$ ; // List of robots initialized with active status
 $\mathcal{I}_{passive} \leftarrow \emptyset$ ; // Set of indexes of all passive robots
for  $t = 1$  to  $T_H$  do
  if deadlock detected among active robots then
    // Find the index of the robot closest to its target
     $i_{closest} \leftarrow \arg \min_{i \in \{1, \dots, M\}} \|p_t^i - p_{target}^i\|_2$ 
    Sort robots  $m^i \in \mathcal{M}$  by decreasing distance to  $m^{i_{closest}}$ 
    // Update  $m^i.\mathcal{I}_{passive}^i$  iteratively, and mark  $m^i$  as passive
    for each robot  $m^i$  in the sorted list of active robots other than  $m^{i_{closest}}$  do
       $m^i.\mathcal{I}_{passive}^i \leftarrow \mathcal{I}_{passive}$   $\mathcal{I}_{passive} = \mathcal{I}_{passive} \cup i$ 
    end
     $m^{i_{closest}}.\mathcal{I}_{passive}^{i_{closest}} \leftarrow \mathcal{I}_{passive}$ ; //  $\star$  Update  $\mathcal{I}_{passive}^{i_{closest}}$  for  $m^{i_{closest}}$   $\star$ 
  end
  if all active robots have arrived at target positions then
    // Re-activate robots based on the distance to targets
     $i_{closest} \leftarrow \arg \min_{i \in \mathcal{I}_{passive}} \|p_t^i - p_{target}^i\|_2$   $\mathcal{I}_{passive} = \mathcal{I}_{passive} \setminus i_{closest}$ 
    Set  $m^{i_{closest}}$  status to active  $m^i.\mathcal{I}_{passive}^i = m^i.\mathcal{I}_{passive}^i \setminus i_{closest} \forall i \in \mathcal{I}_{passive}$ 
  end
end

```

We keep the robot that is closest to its target be active when detect deadlocks and continue to select robots to be passive until the deadlock is resolved.

For a multi-robot systems with n robots, we at most need to select $n-1$ robots to be passive and the remaining active robot is guaranteed to arrive at its target when ignoring all other robots.

But this is inefficient, we propose to use Neural Logic Machines to make these selections

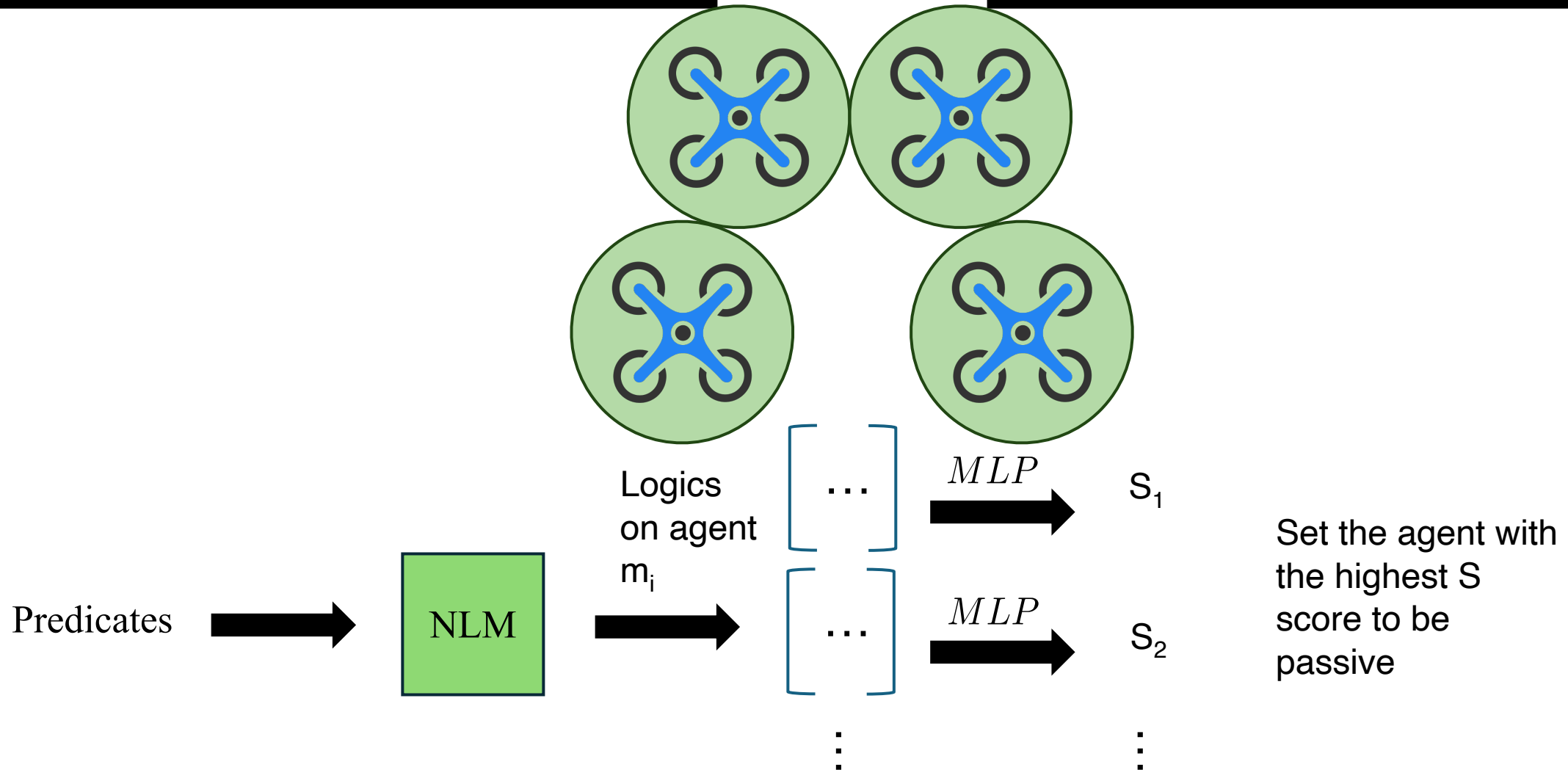
Deadlocks Resolution

Algorithm 1 Greedy Algorithm for Deadlock Resolution

```
 $T_H$  ; // Full Time horizon considered
 $\mathcal{M} = \{m^1, \dots, m^M\}$  ; // List of robots initialized with active status
 $\mathcal{I}_{passive} \leftarrow \emptyset$  ; // Set of indexes of all passive robots
for  $t = 1$  to  $T_H$  do
  if deadlock detected among active robots then
    // Find the index of the robot closest to its target
     $i_{closest} \leftarrow \arg \min_{i \in \{1, \dots, M\}} \|p_t^i - p_{target}^i\|_2$ 
    Sort robots  $m^i \in \mathcal{M}$  by decreasing distance to  $m^{i_{closest}}$ 
    // Update  $m^i.\mathcal{I}_{passive}^i$  iteratively, and mark  $m^i$  as passive
    for each robot  $m^i$  in the sorted list of active robots other than  $m^{i_{closest}}$  do
      |  $m^i.\mathcal{I}_{passive}^i \leftarrow \mathcal{I}_{passive}$   $\mathcal{I}_{passive} = \mathcal{I}_{passive} \cup i$ 
    end
     $m^{i_{closest}}.\mathcal{I}_{passive}^{i_{closest}} \leftarrow \mathcal{I}_{passive}$  ; //  $\star$  Update  $\mathcal{I}_{passive}^{i_{closest}}$  for  $m^{i_{closest}}$   $\star$  /
  end
  if all active robots have arrived at target positions then
    // Re-activate robots based on the distance to targets
     $i_{closest} \leftarrow \arg \min_{i \in \mathcal{I}_{passive}} \|p_t^i - p_{target}^i\|_2$   $\mathcal{I}_{passive} = \mathcal{I}_{passive} \setminus i_{closest}$ 
    Set  $m^{i_{closest}}$  status to active  $m^i.\mathcal{I}_{passive}^i = m^i.\mathcal{I}_{passive}^i \setminus i_{closest} \forall i \in \mathcal{I}_{passive}$ 
  end
end
```

After all active robots have arrived their targets, we set passive robots to become active based on their distance to their own target one at a time

Resolution to the deadlock situation



Resolution to the deadlock situation

Predicates:

<i>FreeToMoveFront</i> (m_i)	True if agent m_i is able to move along its front direction	<i>InDeadlock</i> (m_i, m_j)	True if agent m_j blocked m_i to move for any of the four directions
<i>FreeToMoveBack</i> (m_i)	True if agent m_i is able to move along its back direction	<i>TargetCloser</i> (m_i, m_j)	True if the distance agent m_j to m_j 's target is closer m_i to m_i 's target
<i>FreeToMoveRight</i> (m_i)	True if agent m_i is able to move along its right direction	<i>HigherPriority</i> (m_i, m_j)	True if m_j has a higher priority than m_i
<i>FreeToMoveLeft</i> (m_i)	True if agent m_i is able to move along its left direction		

Action:

<i>SetPassive</i> (m_i)	Set agent m_i to passive status
-----------------------------	-----------------------------------

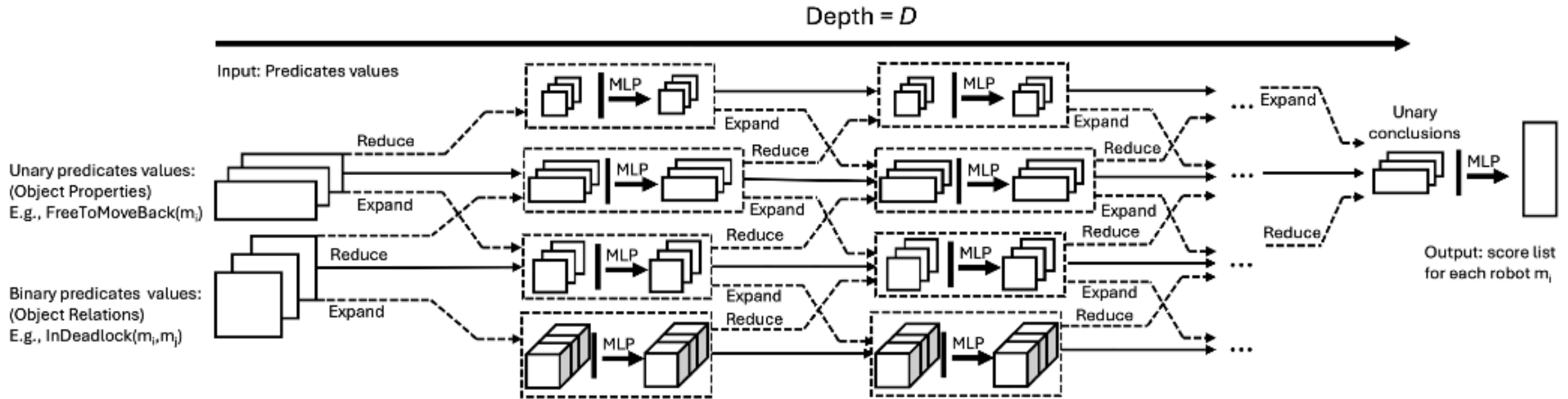


Figure 2: The NLM Pipeline used to select a robot switching to passive status

Algorithm 2 NSDR Algorithm for Deadlock Resolution

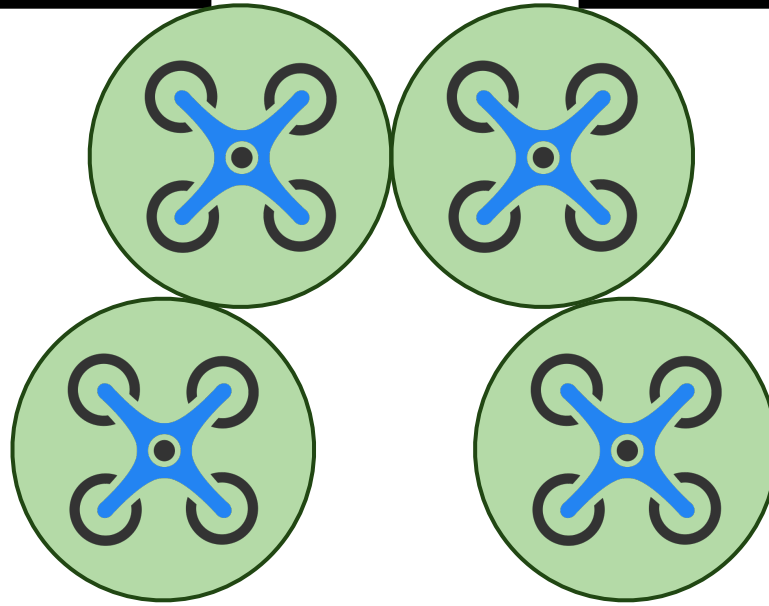
```

 $T_H$ ; // Full time horizon considered
 $\mathcal{M} = \{m^1, \dots, m^M\}$ ; // List of robots initialized with active status
 $\mathcal{I}_{passive} \leftarrow \emptyset$ ; // Set of indexes of all passive robots
for  $t = 1$  to  $T_H$  do
    if deadlock detected among active robots then
        // Evaluate predicate values for  $\mathcal{M}$  and obtain ordered list
        based on NLM scores
         $P_{t,unary}^{\mathcal{M}}, P_{t,binary}^{\mathcal{M}} \leftarrow \text{predicate\_cal}(\mathcal{M})$ 
         $\text{score\_list} \leftarrow \text{NLM}(P_{t,unary}^{\mathcal{M}}, P_{t,binary}^{\mathcal{M}})$ 
        // Select highest feasible scored active robot as  $i_{smax}$ 
        for  $i$  in  $\text{score\_list}$  do
            if selection of robot  $i$  is feasible and  $m^i.status == \text{active}$  then
                 $i_{smax} \leftarrow i$  break
            end
        end
         $\mathcal{I}_{passive} \leftarrow \mathcal{I}_{passive} \cup \{i_{smax}\}$ ; /* Update  $\mathcal{I}_{passive}$  */
        for each robot  $m^i \in \mathcal{M} \setminus m^{i_{smax}}$  and  $m^i.status == 'active'$  do
             $m^i.\mathcal{I}_{passive}^i \leftarrow \mathcal{I}_{passive}$ ; /* Update  $\mathcal{I}_{passive}^i$  for all active robots */
        end
    end
    if all active robots have arrived at target positions then
        // Re-activate robots based on distance to targets
         $i_{closest} \leftarrow \arg \min_{i \in \mathcal{I}_{passive}} \|p_t^i - p_{target}^i\|_2$   $\mathcal{I}_{passive} = \mathcal{I}_{passive} \setminus i_{closest}$ 
        Set  $m_{i_{closest}}$  status to active  $m^i.\mathcal{I}_{passive}^i = m^i.\mathcal{I}_{passive}^i \setminus i_{closest} \forall i \in \mathcal{I}_{passive}$ 
    end
end

```

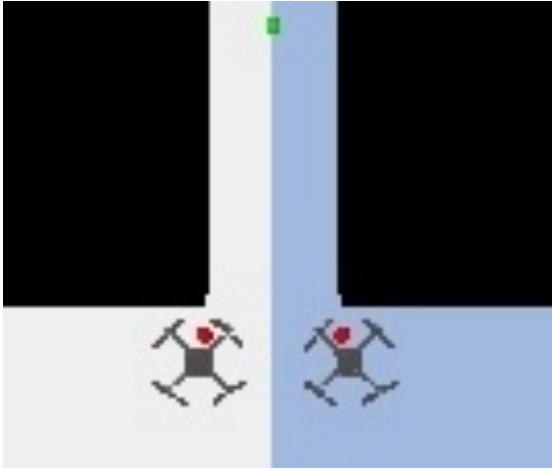
1. Select passive robots based on the score from NLM.
2. Check whether the selection is feasible
3. Update passive robots list for each robot.
4. After all active robots have arrived at their target position, we reactivate passive robots based on their distance to targets.

Resolution to the deadlock situation

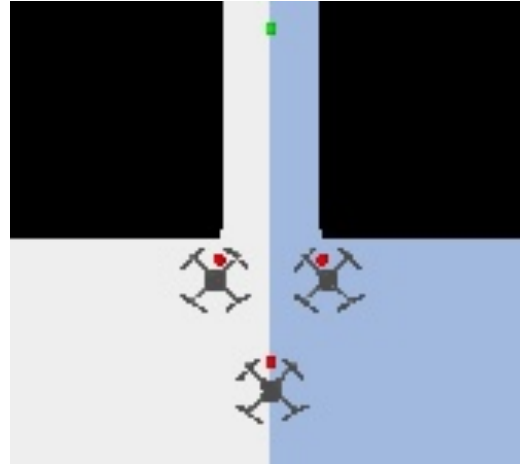


When trained on a small world (involving 2 to 5 agents?), the NLM can learn the logic and generalize to play the game over much larger worlds (involving 10 agents?)

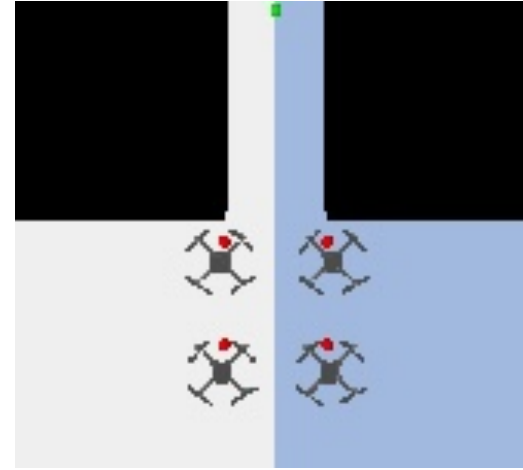
Training Scenarios



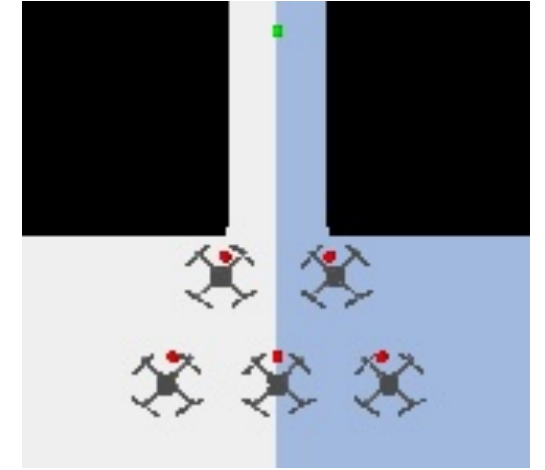
Num robots = 2



Num robots = 3



Num robots = 4



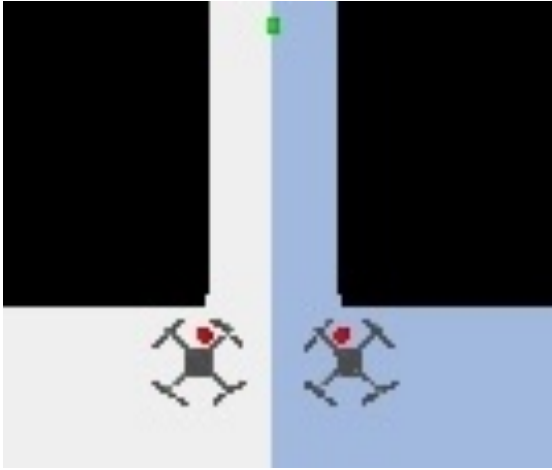
Num robots = 5

For each scenario, we generate all possible selections of passive robots and recorded the arrival time of each agent.

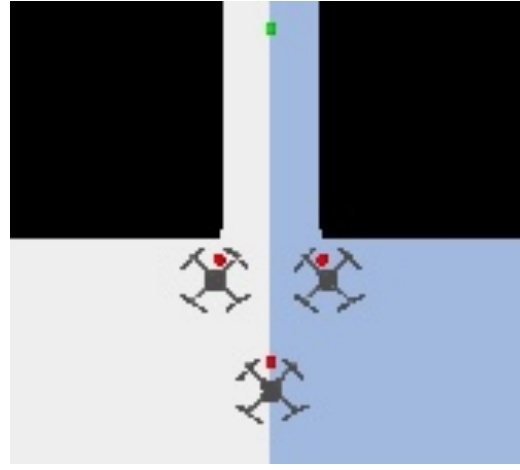
We also augmented the data by including all possible orders of assigning agents to deadlock positions

We also augmented the data further by including 5 different random priorities by randomly assign priorities of 1 or 2 each agent.

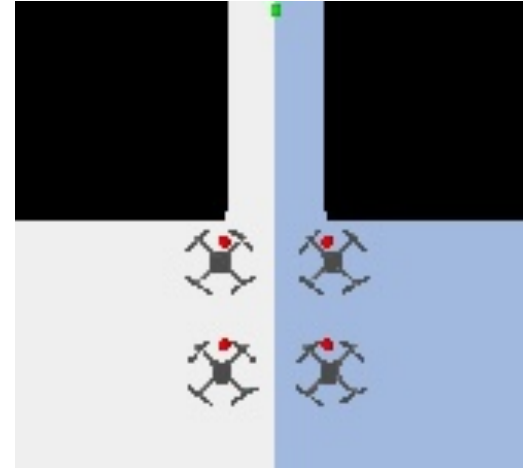
Training Scenarios



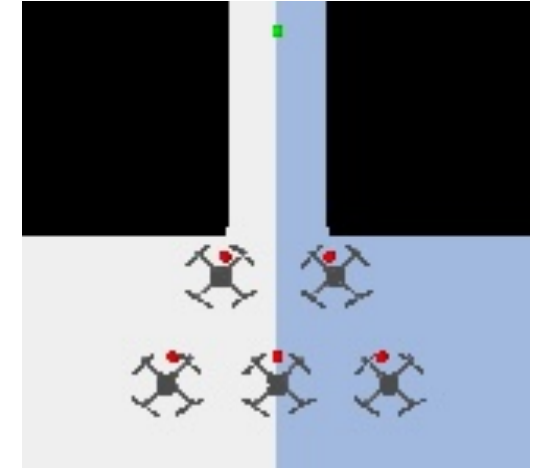
Num robots = 2



Num robots = 3



Num robots = 4



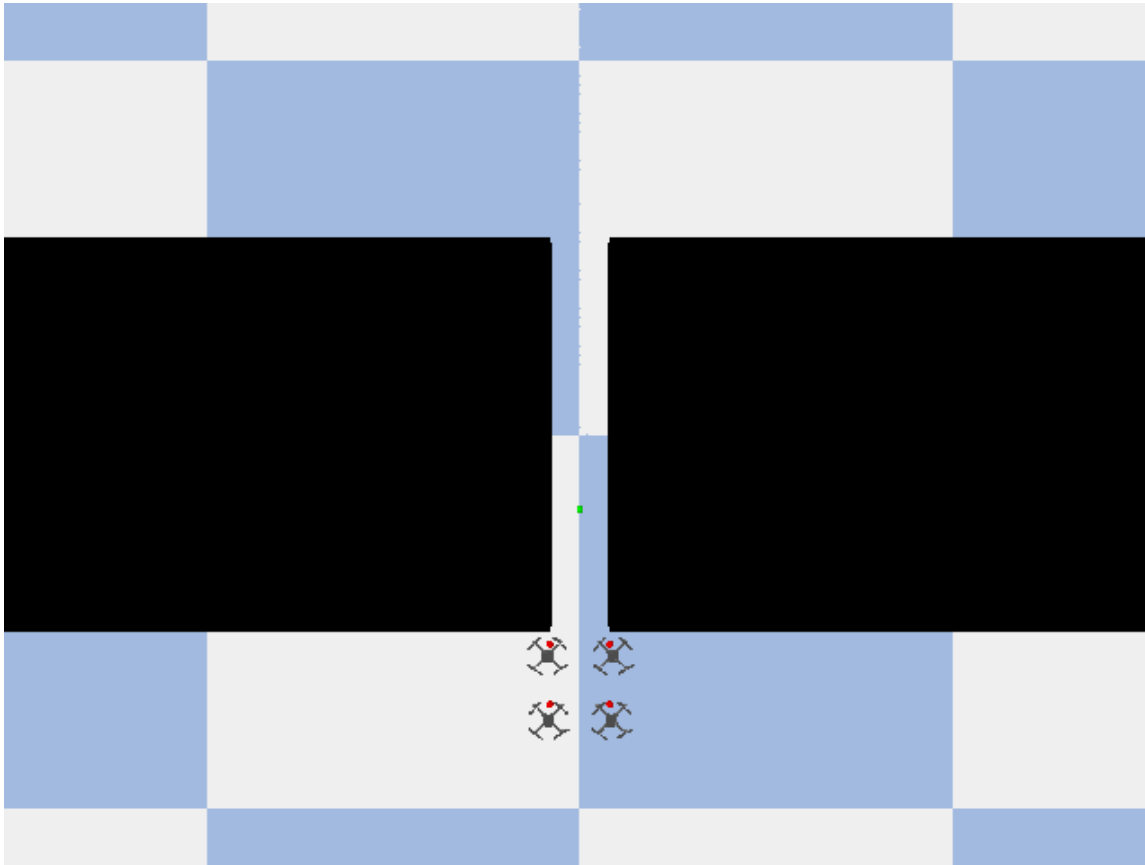
Num robots = 5

We obtained 4299 number of different sets of predicates along with the best passive agent selected that minimized the weighted sum of arrival times of all agents based on a set of predicates

We then divide the whole dataset into 5 folds and performed a supervised 5-fold cross validation training

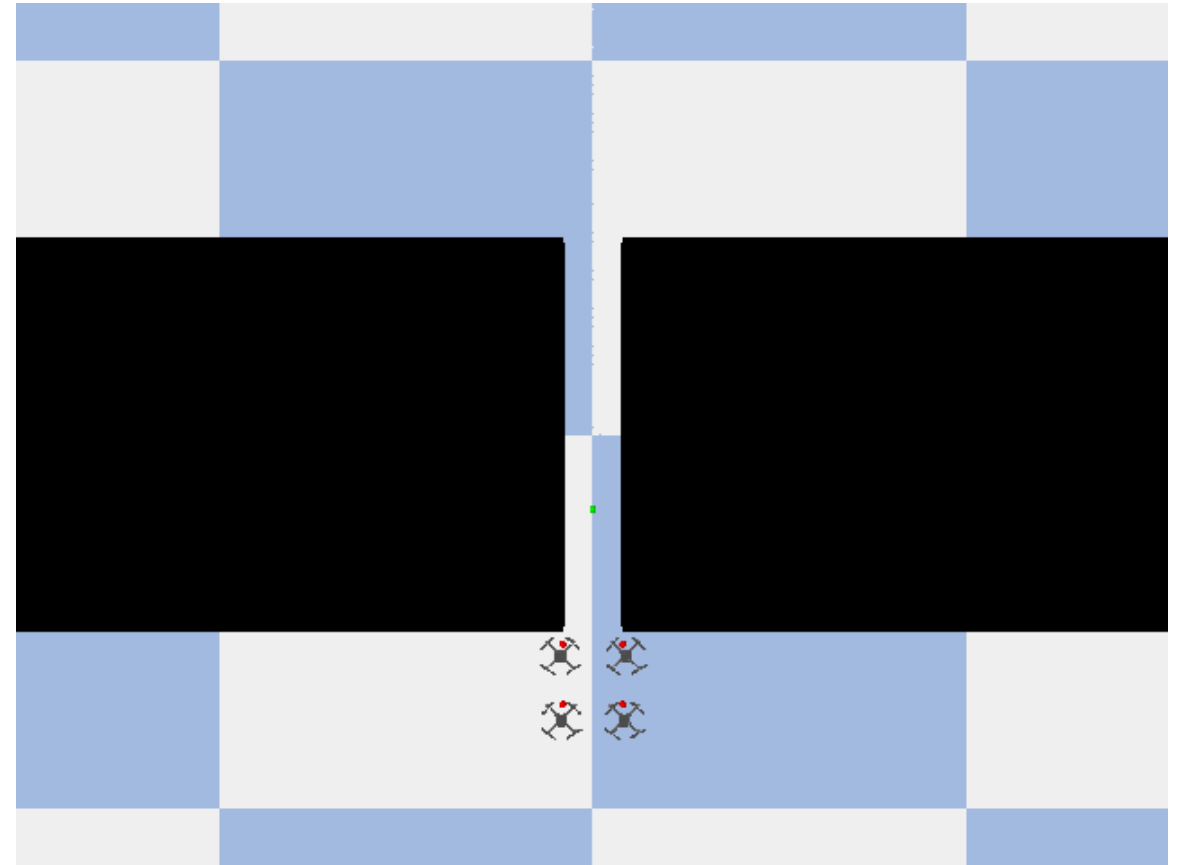
The resulting model can achieve 99.74% training accuracy and 99.55% validation accuracy

In Sample Test



Greedy Method

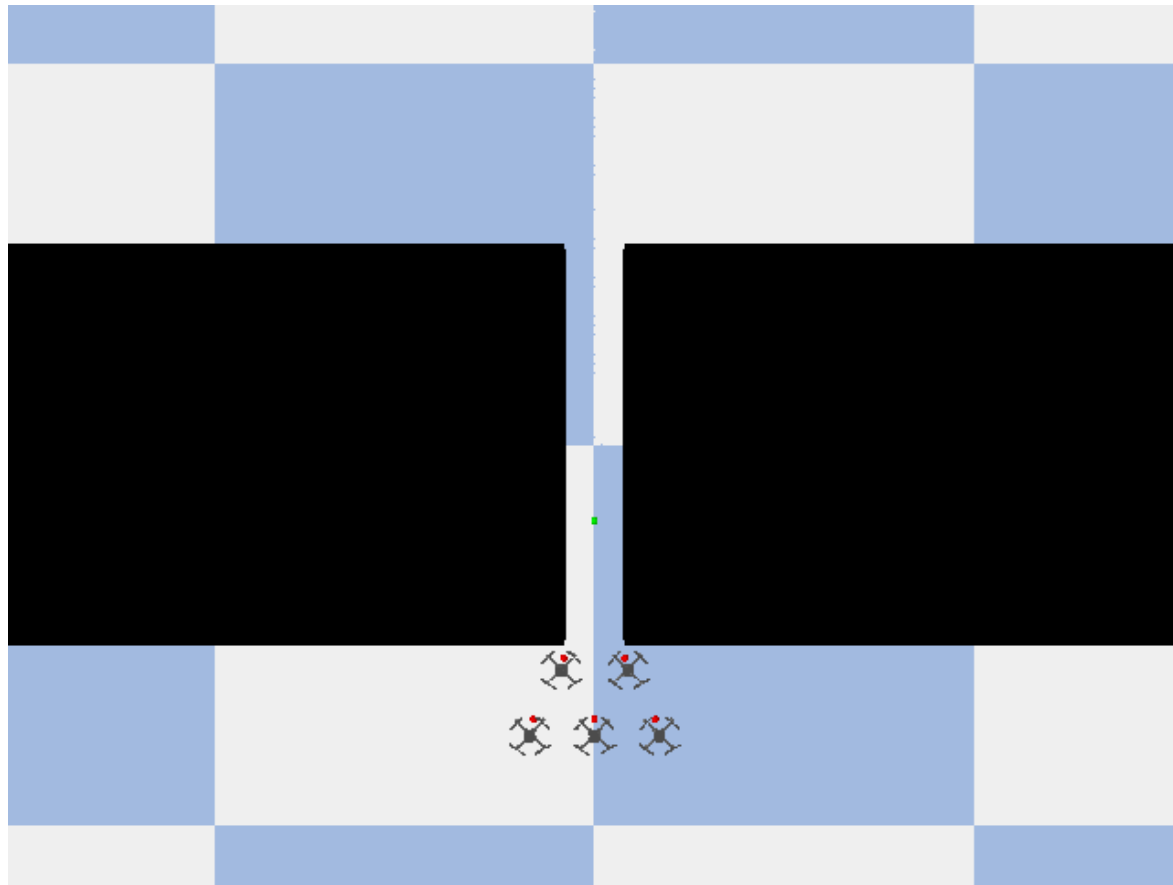
Completion Time: 218 steps



NLM Method

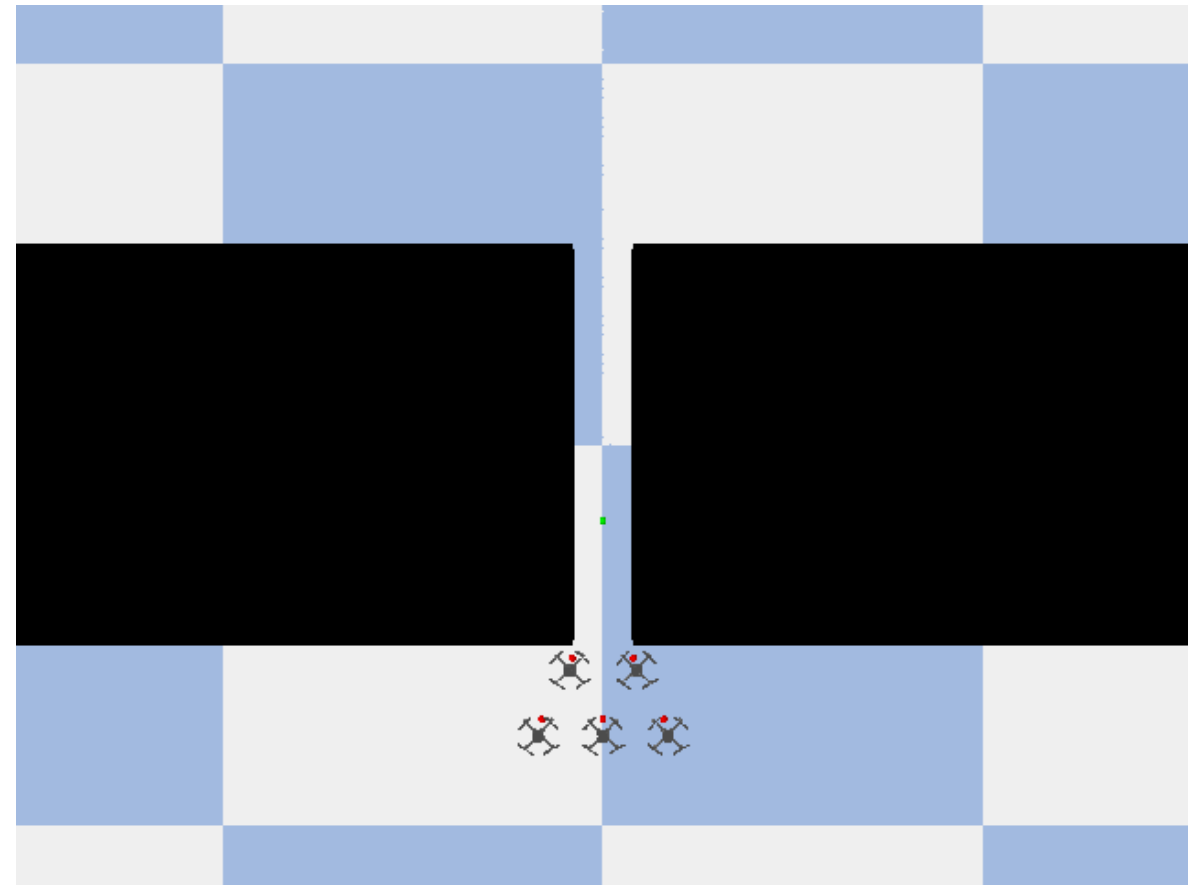
Completion Time: 209 steps

In Sample Test



Greedy Method

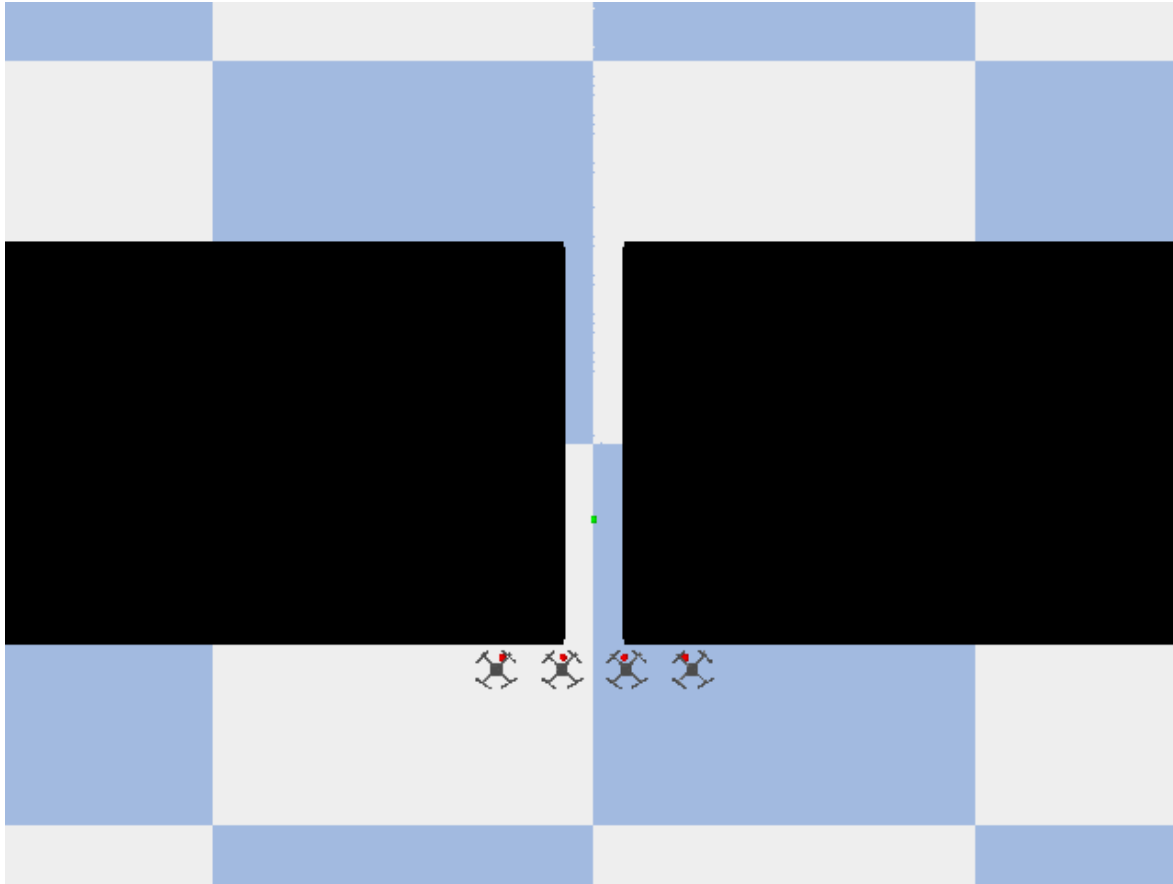
Completion Time: 282 steps



NLM Method

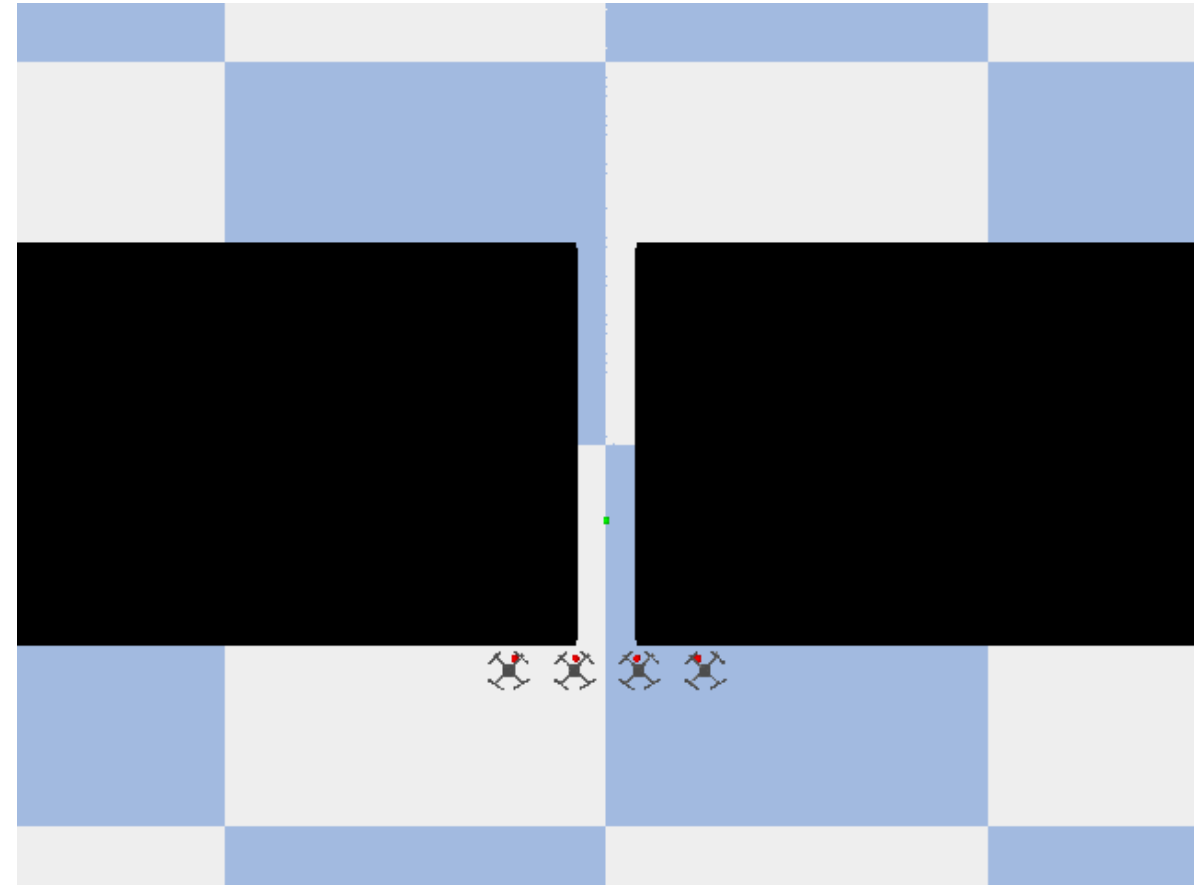
Completion Time: 221 steps

Out Sample Test



Greedy Method

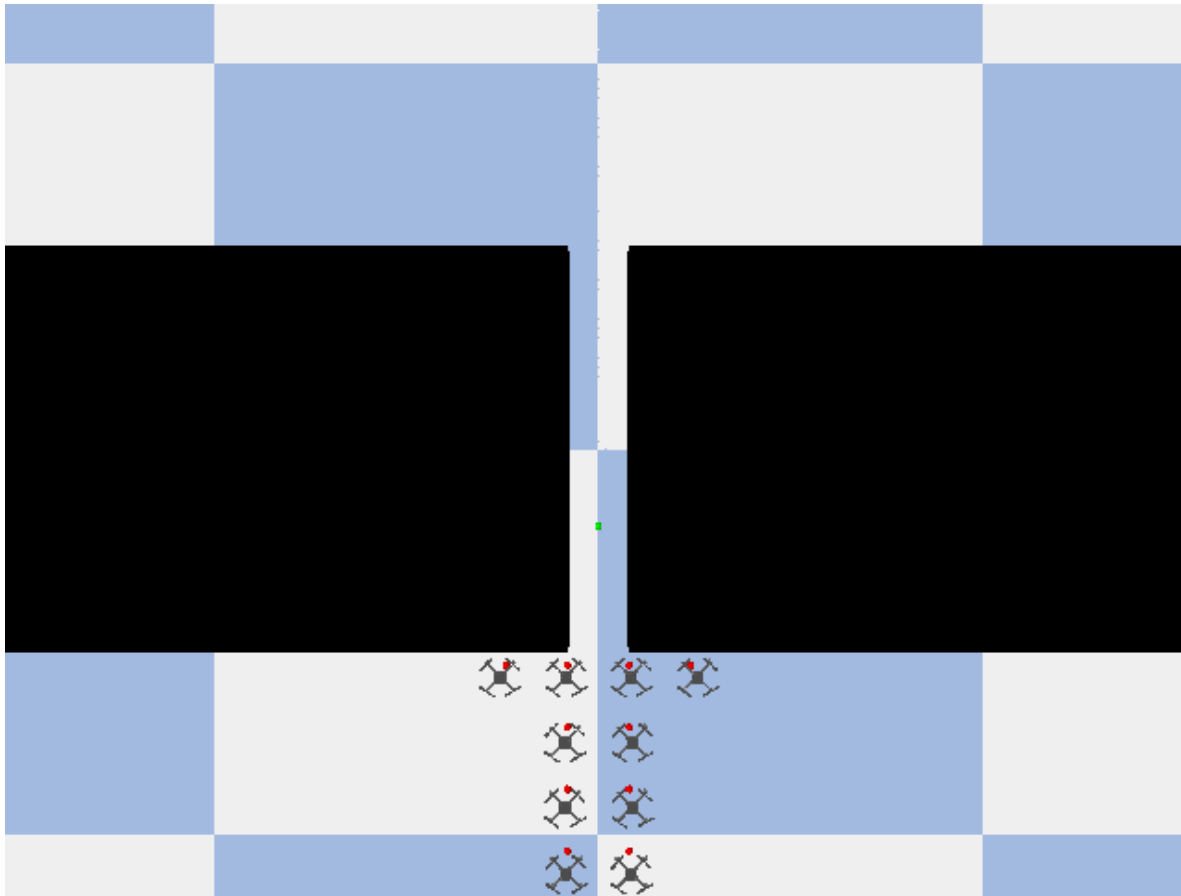
Completion Time: 230 steps



NLM Method

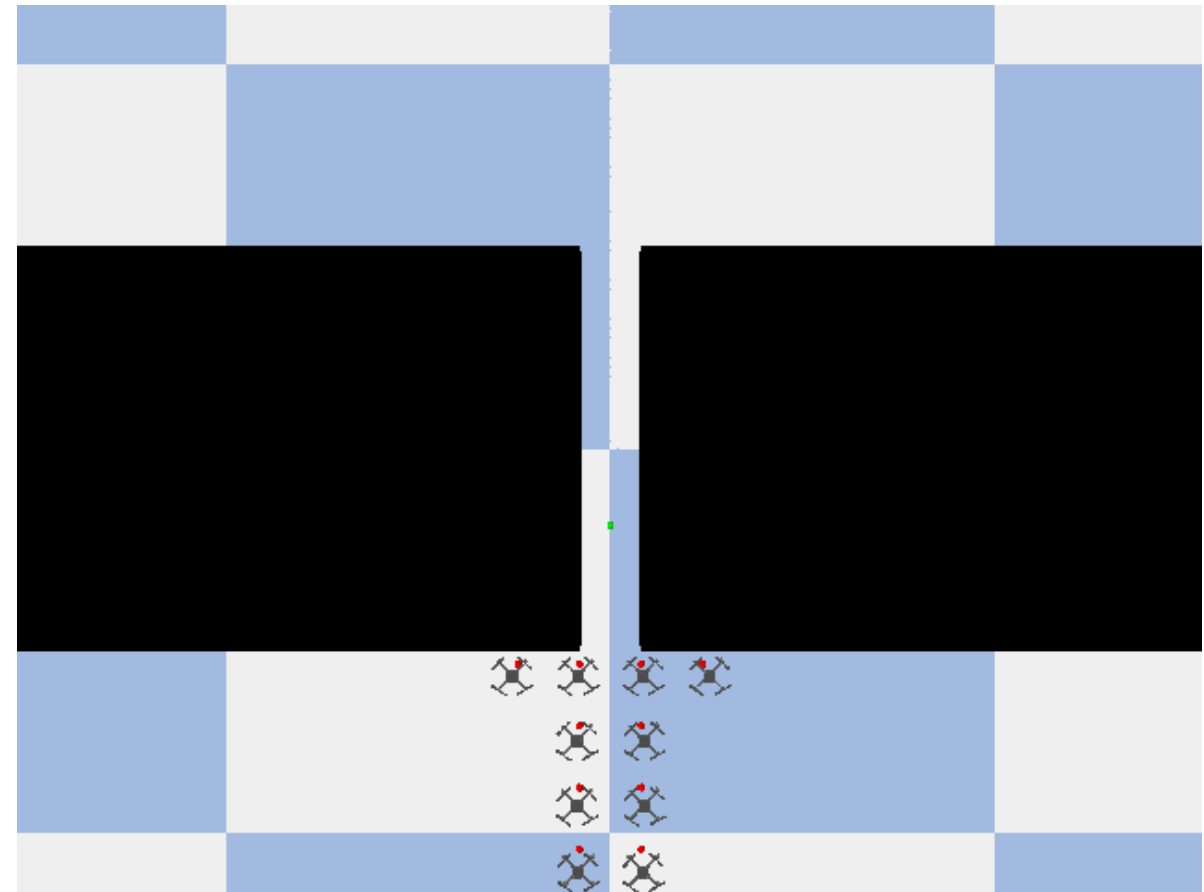
Completion Time: 202 steps

Out Sample Test



Greedy Method

Completion Time: 696 steps



NLM Method

Completion Time: 573 steps